

1 Introduction

Artificial intelligence is now a central part of our lives. Understanding how neural networks work is becoming increasingly important, even for individuals without a background in computer science. Considering AI systems as black boxes can lead to a passive and potentially dangerous relationship with these technologies. Large language models are complex systems that are difficult to understand. In this project, we decided to implement a simple neural network: the perceptron, using 6502 assembly language for the Commodore 64.

In machine learning, the perceptron is an algorithm used for supervised learning of binary classifiers. A binary classifier is a function that determines whether an input, represented as a vector of numbers, belongs to a specific class.¹ The perceptron implemented in this project will be trained to emulate an AND gate. To better understand the algorithm's specifications and the challenges we will encounter during implementation, we will first develop a version of the perceptron in Python. We will then translate this implementation into Commodore 64 assembly language.

We chose to implement the perceptron on the Commodore 64 for several reasons. First, we wanted to explore the architecture of a simple CPU and better understand the constraints and possibilities of low-level programming, which allows close interaction with computer hardware. The Commodore 64 was an interesting choice because of its limited instruction set, making it a suitable platform for studying the fundamentals of neural network implementation at a low level.

2 Theoretical background

A perceptron is a learning algorithm used for binary classification. It is the fundamental computational unit of a neural network and can be viewed as a function that maps an input vector $\mathbf{x}$, whose components are real-valued numbers, to an output $f(\mathbf{x})$.² To perform binary classification, the perceptron must learn a decision boundary that separates the two classes. In the two-dimensional case considered in this project, this decision boundary is a straight line (see Fig. 2). The perceptron uses a linear model in which each input is multiplied by an associated weight, and a bias term is added: $z = w_1x_1 + w_2x_2 + b$

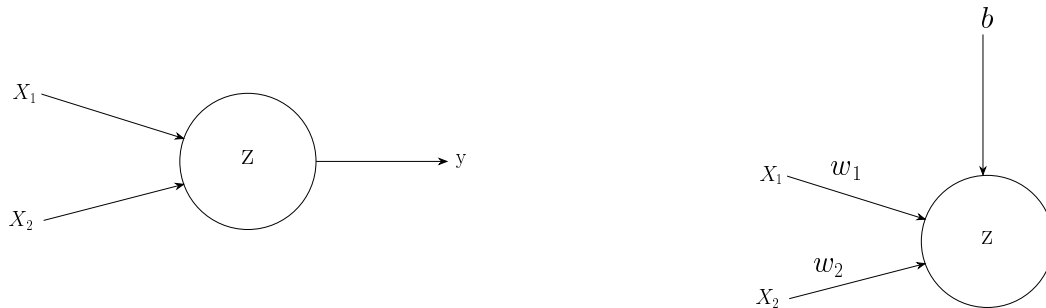


Figure 1: Perceptron figure and linear model with 2 inputs

Before being used for prediction, the perceptron is trained on a dataset. The objective of the training process is to determine the optimal values of the weights and the bias so as to minimize the error between the expected output and the prediction produced by the model. To quantify this error, we use the hinge loss function (described below), which assigns a cost to each prediction according to the expected label. The model parameters are then optimized using the gradient descent algorithm, which iteratively updates the weights and the bias to minimize the loss function.

Once the perceptron has been trained, it can classify previously unseen data. Given a new input, the linear model computes the value of z , which indicates on which side of the decision boundary the input lies. Finally, the step activation function converts this value into a binary prediction corresponding to one of the two classes.

2.1 Cost function and Activation function

¹Freund, Y.; Schapire, R. E. (1999). "Large margin classification using the perceptron algorithm"

²Wikipedia contributors. "Perceptron." Wikipedia, The Free Encyclopedia. Wikipedia, The Free Encyclopedia, 22 Jun. 2026. Web. 18 Jul. 2026.

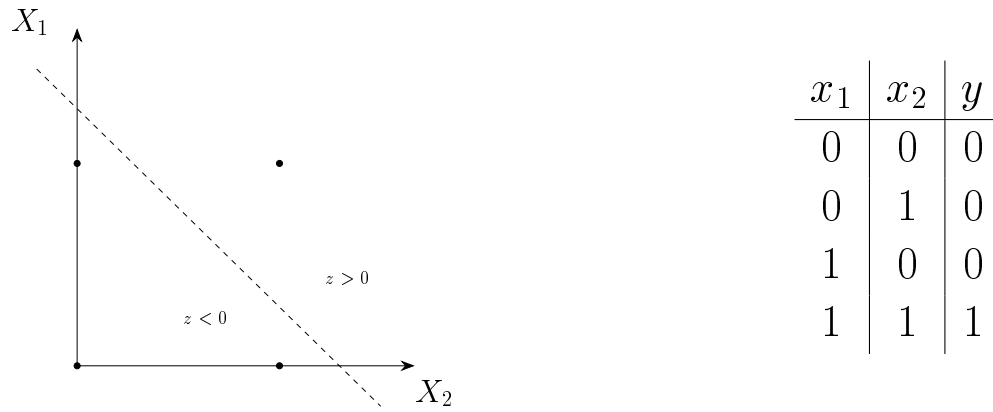


Figure 2: Truth table of AND gate and linear separation between the two classes.

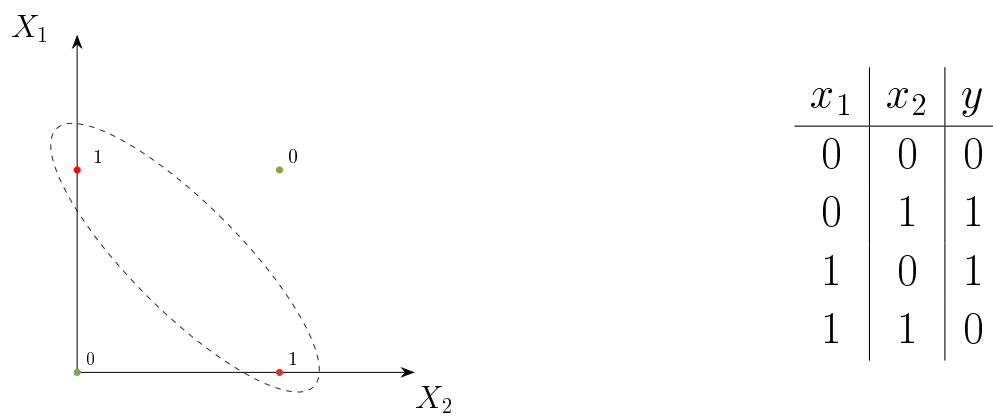


Figure 3: Truth table of XOR gate and non-linear separation between the two classes.